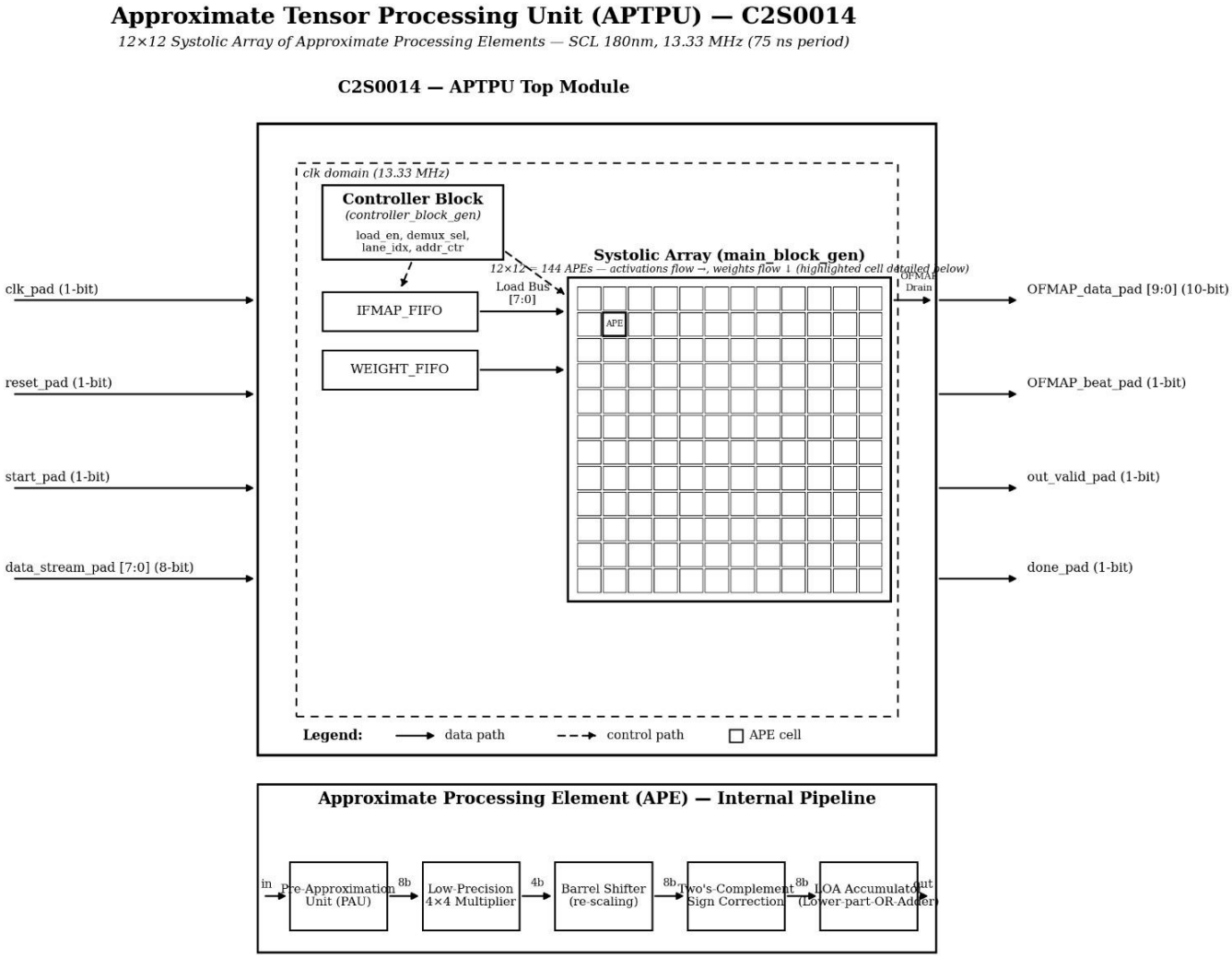
Approximate Computing Based Tensor Processing Unit (APTPU)

What is an APTPU?

An **APTPU (Approximate Computing Based Tensor Processing Unit)** is a dedicated AI hardware accelerator that performs tensor and matrix computations efficiently using approximate arithmetic techniques. It is optimized for deep learning workloads, providing reduced power consumption, lower hardware area, and faster execution while maintaining high computational performance for neural network inference.

Block Architecture:

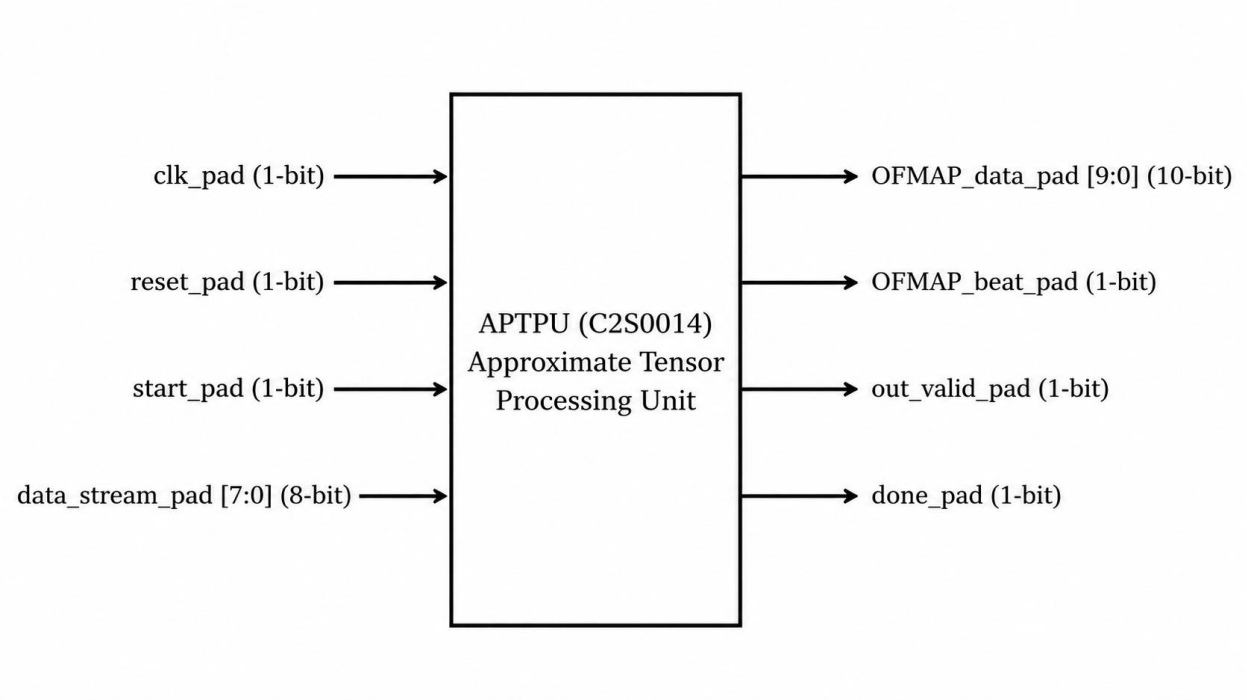
The figure illustrates the overall architecture of the **Approximate Tensor Processing Unit (APTPU)**. It consists of a **Controller Block**, **IFMAP FIFO**, **Weight FIFO**, and a **12 × 12 Approximate Processing Element (APE) Systolic Array**. The controller manages data flow, while the FIFOs supply input feature maps and weights to the systolic array. Each APE performs approximate multiply-accumulate operations using a PAU, low-precision multiplier, barrel shifter, sign correction, and LOA accumulator. The processed output is then transferred to the OFMAP output interface, enabling high-throughput and energy-efficient tensor processing



Description:

The APTPU architecture is designed to efficiently accelerate tensor computations for deep learning applications. The controller coordinates data flow between the IFMAP and Weight FIFOs and the 12×12 systolic array. Each Approximate Processing Element (APE) performs low-power approximate multiply-accumulate operations using a PAU, low-precision multiplier, barrel shifter, sign correction, and LOA accumulator. This architecture achieves high throughput while reducing power consumption and hardware area.

BLOCK DIAGRAM BOX:



The black box diagram represents the top-level interface of the **Approximate Tensor Processing Unit (APTPU)**. The module receives **clock (clk_pad)**, **reset (reset_pad)**, and **start (start_pad)** signals to control its operation, while the **data_stream_pad[7:0]** serves as the 8-bit input data bus for tensor processing. Internally, the APTPU performs approximate tensor computations using its dedicated processing architecture. After computation, the processed output feature map is available through **OFMAP_data_pad[9:0]**, while **OFMAP_beat_pad**, **out_valid_pad**, and **done_pad** provide output synchronization, data validity, and computation completion status, respectively. This interface enables efficient communication between the APTPU and external hardware modules during AI inference.

Features:

- Supports **high-speed tensor and matrix computations** for AI applications.
- Uses **approximate computing** techniques to reduce power consumption.
- **Low hardware area** due to simplified arithmetic operations.
- **High-throughput** processing using a systolic array architecture.
- Includes **FIFO buffers** for efficient data movement.
- Controlled by a **Finite State Machine (FSM)** for synchronized operation.

Real-Time Applications:

- **AI Inference Acceleration:** Efficient execution of deep learning workloads for neural network inference.
- **Edge AI Devices:** Enables low-power AI processing in smart cameras, drones, robots, and IoT devices.
- **Computer Vision:** Accelerates image recognition, object detection, and classification tasks in real time.
- **Embedded AI Systems:** Provides high-performance, energy-efficient tensor computations for ASIC and edge computing applications.

INPUT/OUTPUT PORTS:

MODULE: C2S0014 (Systolic Array Accelerator, M=12)

INPUT PORTS (I/P)

S. No.	Port Name	Width (Bits)	Description
1.	clk	1	System clock input
2.	reset	1	Active high reset
3.	start	1	Start signal for operation
4.	data_stream	8	Shared load bus carrying IFMap words during LOAD_IF phase and Weight words during LOAD_W phase.
TOTAL INPUT PORTS			11

OUTPUT PORTS (O/P)

S. No.	Port Name	Width (Bits)	Description
1.	OFMAP_data	10	Streaming result output, 2 beats per 20-bit MAC result
2.	OFMAP_beat	1	Beat select: 0=low 10 bits, 1=high 10 bits
3.	out_valid	1	Indicates OFMAP_data output is valid
4.	done	1	Indicates accelerator has completed operation.
TOTAL			13

Total I/O Ports: (11 + 13) = 24

TOTAL PADS CALCULATION (WITH AND WITHOUT POWER PADS)

WITHOUT POWER PADS

Description	Calculation	Total Pads
I/O Pads (Only)	Total I/O Ports = 24	**24 Pads**

WITHOUT POWER PADS : 24 PADS

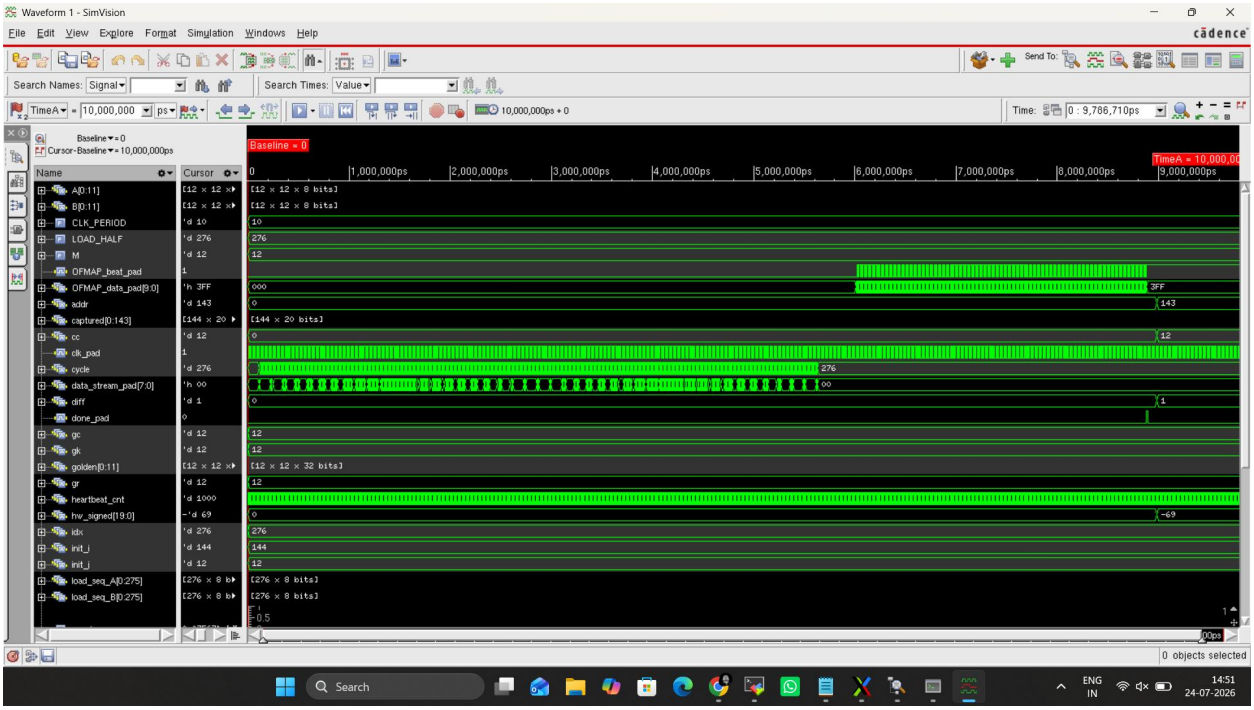
WITH POWER PADS

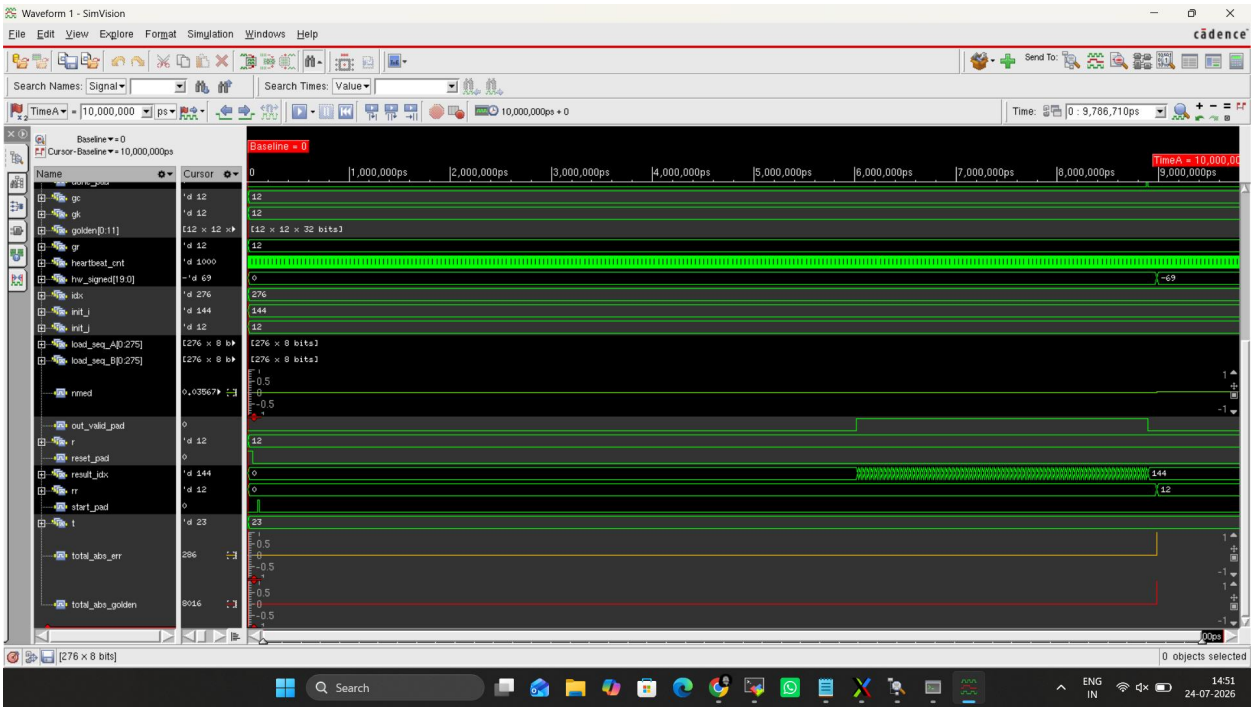
Description	Calculation	Total Pads
I/O Pads	Total I/O Ports = 24	24
Power Pads (All 4 Sides)	Ratio I/O : Pwr = 6 24/6 = 4 per side 4 x 4 sides = 16 Power Pads	16
Total Pads (with Power Pads)		40

WITH POWER PADS : 41 PADS
(24 I/O Pads + 16 Power Pads + 1 Buffer Clock)

The Figure presents the input and output port specification of the **C2S0014 Approximate Tensor Processing Unit (APTPU)**. The module receives four input signals: **clk**, **reset**, **start**, and an **8-bit data_stream**, which are used for clock synchronization, initialization, operation control, and input data transfer. The output interface consists of **OFMAP_data (10-bit)**, **OFMAP_beat**, **out_valid**, and **done** signals, which provide the processed output feature map, output beat selection, data validity indication, and operation completion status, respectively. The figure also summarizes the total I/O count and pad calculation used for ASIC implementation, including both I/O pads and power pads required for chip integration.

SIMULATION:





The waveform shown in Figures represents the functional simulation results of the **Approximate Tensor Processing Unit (APTPU)** obtained using **Cadence**

SimVision. Functional simulation was performed to verify the correctness of the RTL design before synthesis and physical implementation. The waveform confirms that the design operates according to the intended functionality under different input and control conditions.

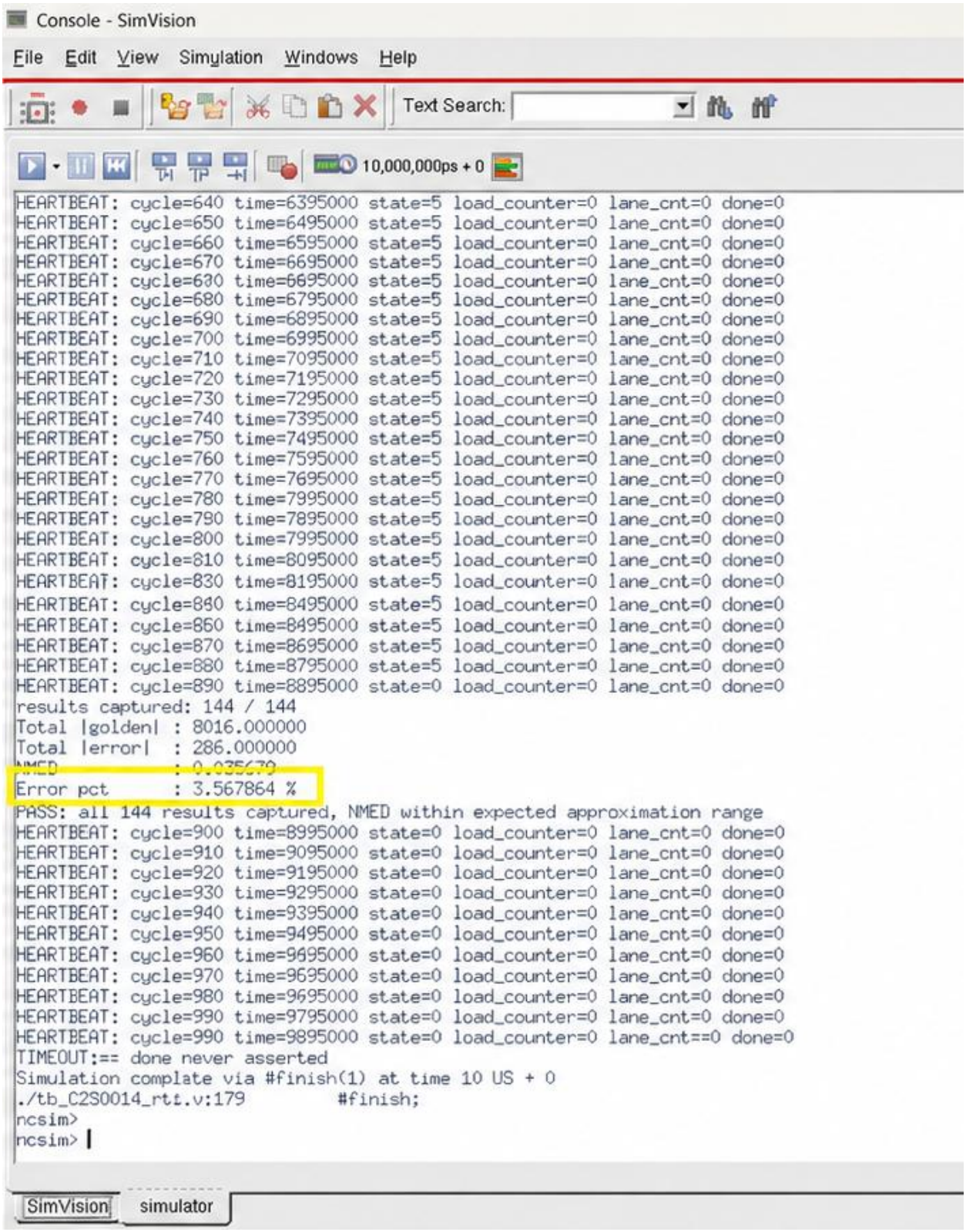
At the beginning of the simulation, the **clock (clk_pad)** provides the synchronization signal for all sequential operations, while the **reset (reset_pad)** initializes the internal registers and processing elements to a known state. After the reset is released, the **start_pad** signal is asserted to begin the tensor processing operation. The **data_stream_pad[7:0]** continuously supplies the input feature map (IFMAP) and weight data required by the systolic array for computation.

The controller coordinates the loading of data into the IFMAP and Weight FIFOs and manages the data flow across the processing elements. As the simulation progresses, the internal counters, load sequences, and result indices update according to the execution stages. The Approximate Processing Elements (APEs) perform approximate multiply-accumulate operations in a pipelined manner, allowing multiple computations to be processed simultaneously. This pipelined execution significantly improves throughput while reducing computational delay.

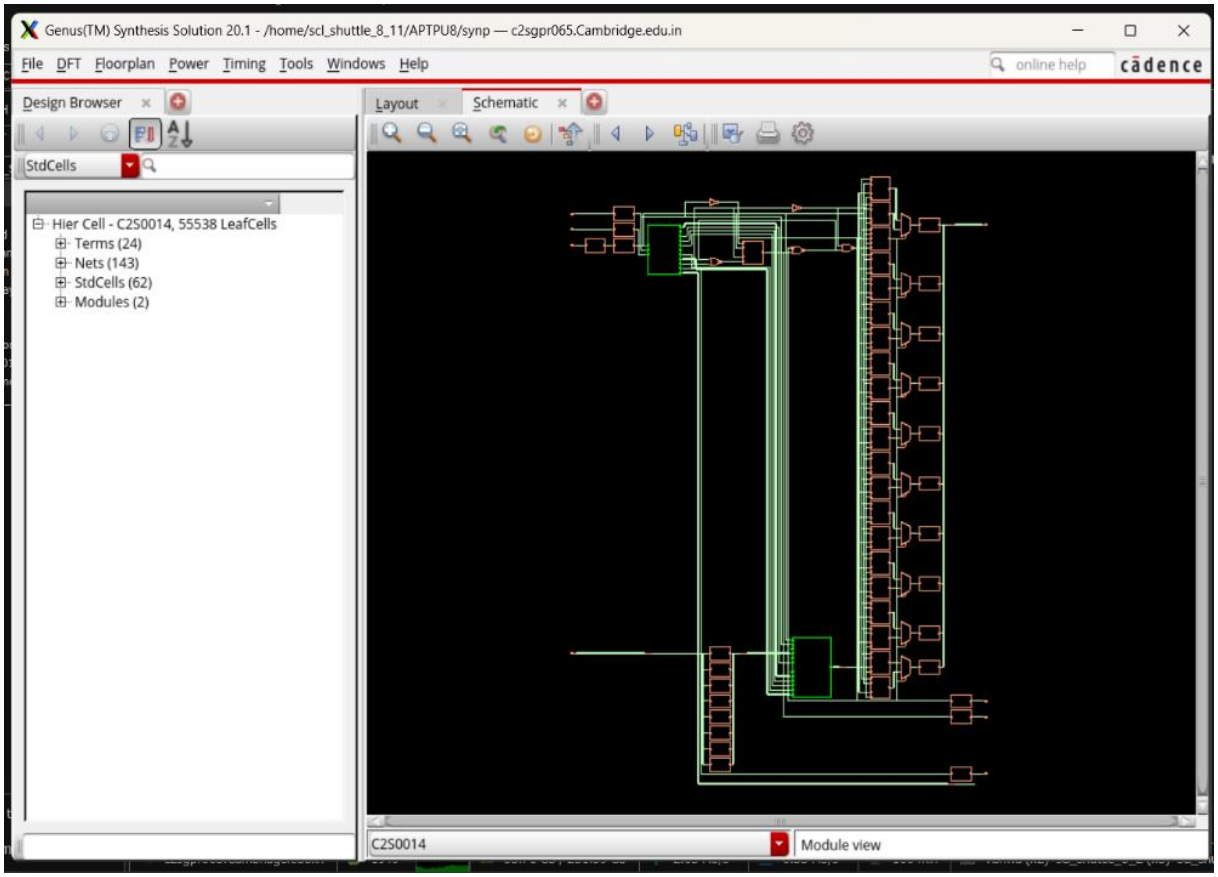
Once the tensor computations are completed, the processed output feature map is generated on the **OFMAP_data_pad[9:0]** output bus. The **out_valid_pad** signal indicates that the output data is valid and ready for external use, while the **OFMAP_beat_pad** controls the transmission of serialized output data. Finally, the **done_pad** signal is asserted, indicating that the accelerator has successfully completed the processing operation.

The simulation results verify the correct functionality of the controller, FIFOs, systolic array, approximate processing elements, and output interface.

TCL CONSOLE ANALYSIS:

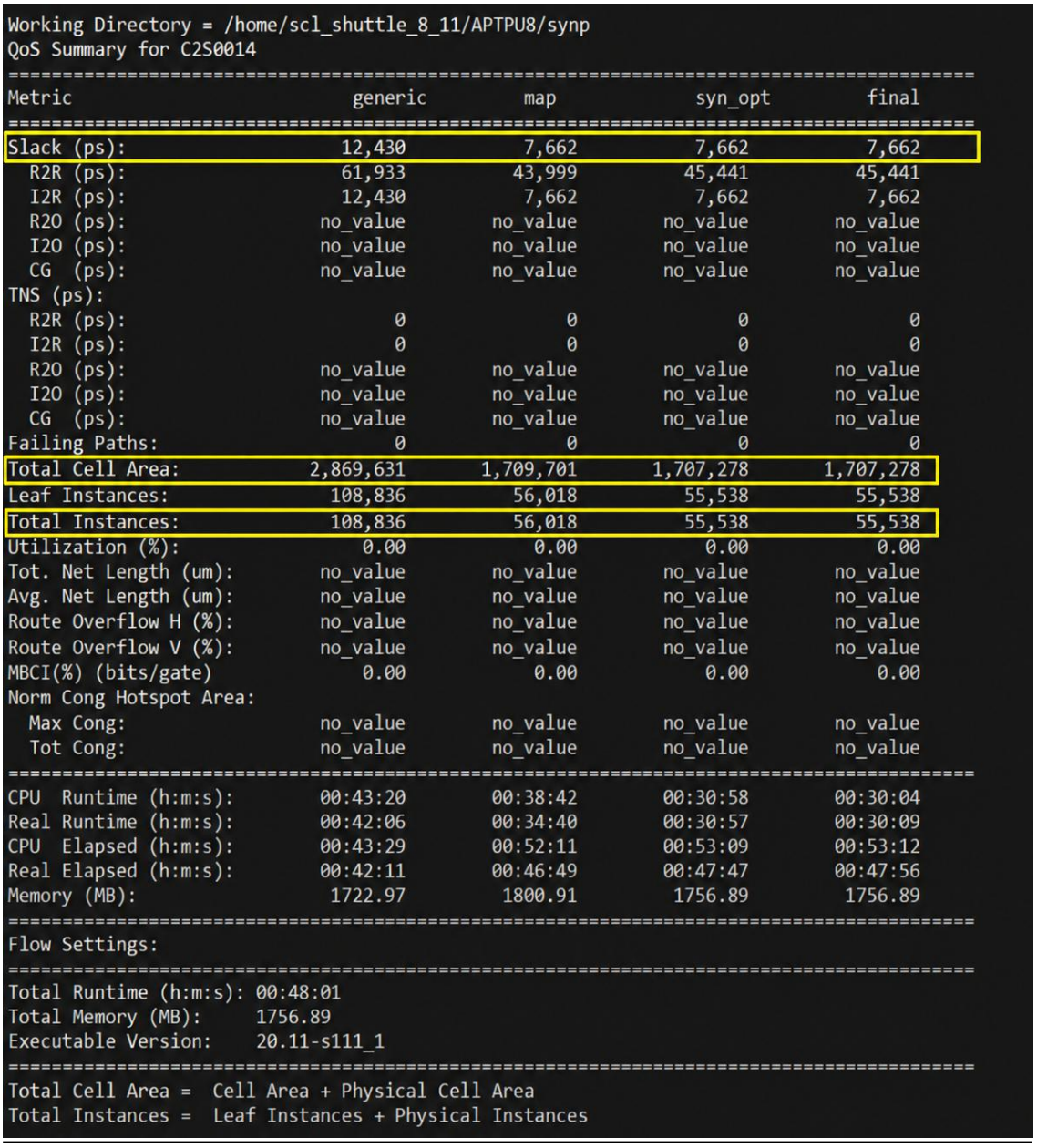


SYNTHESIS:



This figure shows the gate-level schematic of the synthesized C2S0014 design generated using Cadence Genus. The RTL design is mapped to the SCL 180 nm standard cell library, producing a technology-specific gate-level netlist. This synthesized design is used as the input for the physical design stages, including floorplanning, placement, clock tree synthesis (CTS), and routing.

SYNTHETIC REPORT:



```
Working Directory = /home/scl_shuttle_8_11/APTPU8/synp
QoS Summary for C2S0014
================================================================================
Metric                          generic         map          syn_opt        final
================================================================================
Slack (ps):                      12,430       7,662            7,662        7,662
  R2R (ps):                      61,933      43,999           45,441       45,441
  I2R (ps):                      12,430       7,662            7,662        7,662
  R2O (ps):                    no_value    no_value         no_value     no_value
  I2O (ps):                    no_value    no_value         no_value     no_value
  CG  (ps):                    no_value    no_value         no_value     no_value
TNS (ps):
  R2R (ps):                           0           0                0            0
  I2R (ps):                           0           0                0            0
  R2O (ps):                    no_value    no_value         no_value     no_value
  I2O (ps):                    no_value    no_value         no_value     no_value
  CG  (ps):                    no_value    no_value         no_value     no_value
Failing Paths:                        0           0                0            0
Total Cell Area:              2,869,631   1,709,701        1,707,278    1,707,278
Leaf Instances:                 108,836      56,018           55,538       55,538
Total Instances:                108,836      56,018           55,538       55,538
Utilization (%):                    0.00        0.00             0.00         0.00
Tot. Net Length (um):        no_value    no_value         no_value     no_value
Avg. Net Length (um):        no_value    no_value         no_value     no_value
Route Overflow H (%):        no_value    no_value         no_value     no_value
Route Overflow V (%):        no_value    no_value         no_value     no_value
MBCI(%) (bits/gate)                 0.00        0.00             0.00         0.00
Norm Cong Hotspot Area:
  Max Cong:                  no_value    no_value         no_value     no_value
  Tot Cong:                  no_value    no_value         no_value     no_value
================================================================================
CPU  Runtime (h:m:s):        00:43:20    00:38:42         00:30:58     00:30:04
Real Runtime (h:m:s):        00:42:06    00:34:40         00:30:57     00:30:09
CPU  Elapsed (h:m:s):        00:43:29    00:52:11         00:53:09     00:53:12
Real Elapsed (h:m:s):        00:42:11    00:46:49         00:47:47     00:47:56
Memory (MB):                  1722.97     1800.91          1756.89      1756.89
================================================================================
Flow Settings:
================================================================================
Total Runtime (h:m:s): 00:48:01
Total Memory (MB):     1756.89
Executable Version:    20.11-s111_1
================================================================================
Total Cell Area =  Cell Area + Physical Cell Area
Total Instances =  Leaf Instances + Physical Instances
```

This figure shows the synthesis Quality of Results (QoR) summary generated by **Cadence Genus**. It highlights key synthesis metrics, including **timing slack**, **total cell area**, and **total instances**, which are used to evaluate the timing performance, hardware resource utilization, and complexity of the synthesized design before physical implementation.

Clock Period vs Slack Value After Synthesis

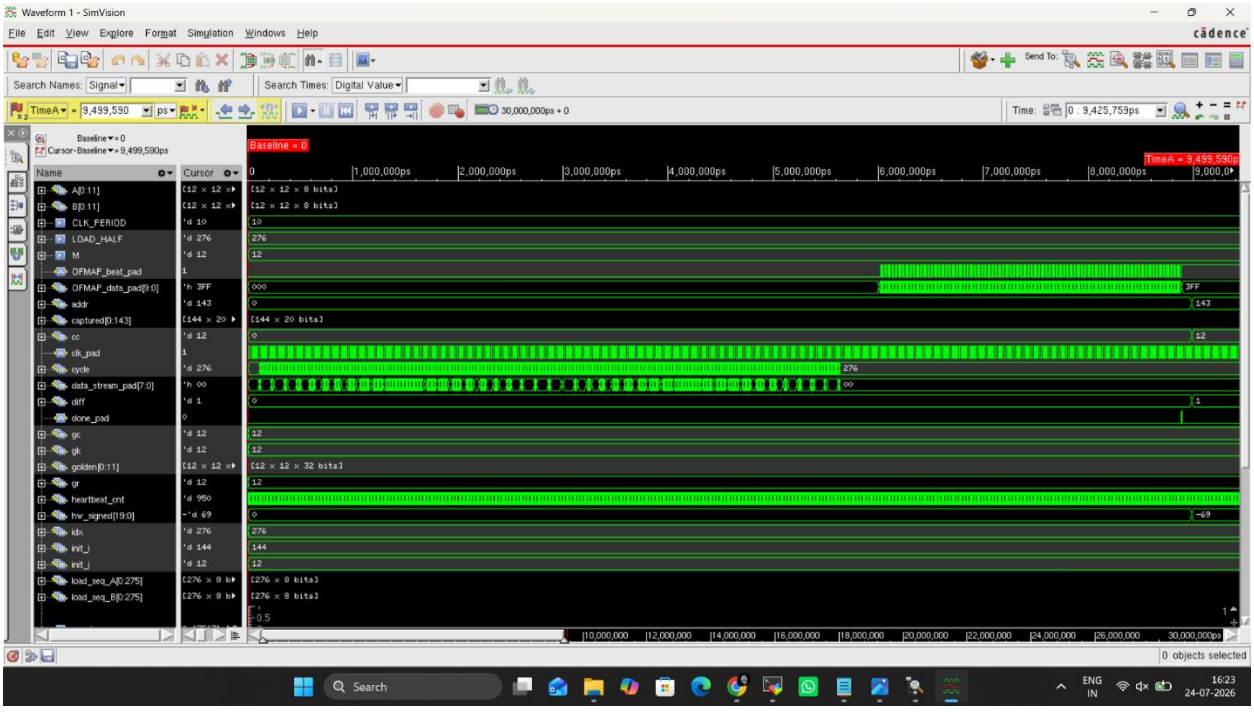
Clock Period (ns)	Slack (ps)
33	-9168
45	-5468
52	-1000
65	-7400
75	7662
89	-12,900
99	-12,999
Matched Condition: Clock Period = 75 ns, Slack = 7662 ps	

This figure presents the synthesis results obtained by evaluating the design under different **clock period constraints**. Several clock periods were tested to analyze their impact on timing slack. Among the evaluated values, a **clock period of 75 ns** achieved a **positive slack of 7662 ps**, satisfying the timing requirements. Therefore, **75 ns** was selected as the final clock period for the synthesis process.

Logical Equivalence Check (LEC):

```
================================================================================
|                    Conformal Version |20.20-s200
|                           Root Module |C2S0014
|                              CPU Time |1516.0 sec
|                          Elapsed Time |13011.0 sec
|                               Memory |894.41 MB
|                   Power Grid Comparison |N/A
|                    Power Intent Compare |N/A
|                Supply Power Consistency |N/A
|              Retention Power Consistency |N/A
|                   Compare Power Crossing |N/A
|                         LEC Comparison |PASS
|                      Flatten Comparison |N/A
|                  Hierarchical Comparison |PASS
|          Hierarchical Comparison Modules |771
|                           Logfile Lines |84828
|                            Logfile Name |rtl2intermediate.lec.log
|                     Logfile Modified Time |Mon Jul 27 14:22:54 2026
|               Implementation Information |fv/C2S0014
|             Set Verification Information |/home/scl_shuttle_8_11/APTPU8/pd/lec2/rtl_fv_map_db
|            Read Verification Information |N/A
|                         Analysis Elapsed |2.3 sec
|                           Dynamic Flatten |0
|                           Runtime / #Gate |0.05 sec.
|          Low-power Rule Check Occurrence |N/A
|Matched back-to-back isolation cell       |N/A
|                  Time-consuming commands | 1. compare: 1435.0s(77.82%) (execute 771 time(s))
|                                          | 2. analyze abort: 144.0s(7.81%) (execute 771 time(s))
|                                          | 3. usage: 109.0s(5.91%) (execute 2 time(s))
```

Gate-Level Simulation (GLS):



ANALYSIS OF LEC AND GLS:

Logical Equivalence Check (LEC)

Logical Equivalence Check (LEC) was performed using **Cadence Conformal** to verify the functional equivalence between the synthesized gate-level netlist generated by Cadence Genus and the post-route netlist generated by Cadence Innovus. The purpose of LEC is to ensure that the optimization and physical design processes, including placement, clock tree synthesis (CTS), and routing, do not introduce any unintended functional changes to the design.

During verification, the synthesized netlist was used as the **golden (reference)** design, while the post-route netlist without power and ground connections was used as the **revised (implementation)** design. The comparison was carried out at the logical level, and all compare points were successfully matched. The verification completed with **zero non-equivalent points, zero abort points, and zero unknown points**, indicating that the post-route implementation is logically identical to the synthesized design.

The successful completion of LEC confirms that the physical implementation has preserved the intended functionality of the design and that the netlist is suitable for signoff and further validation.

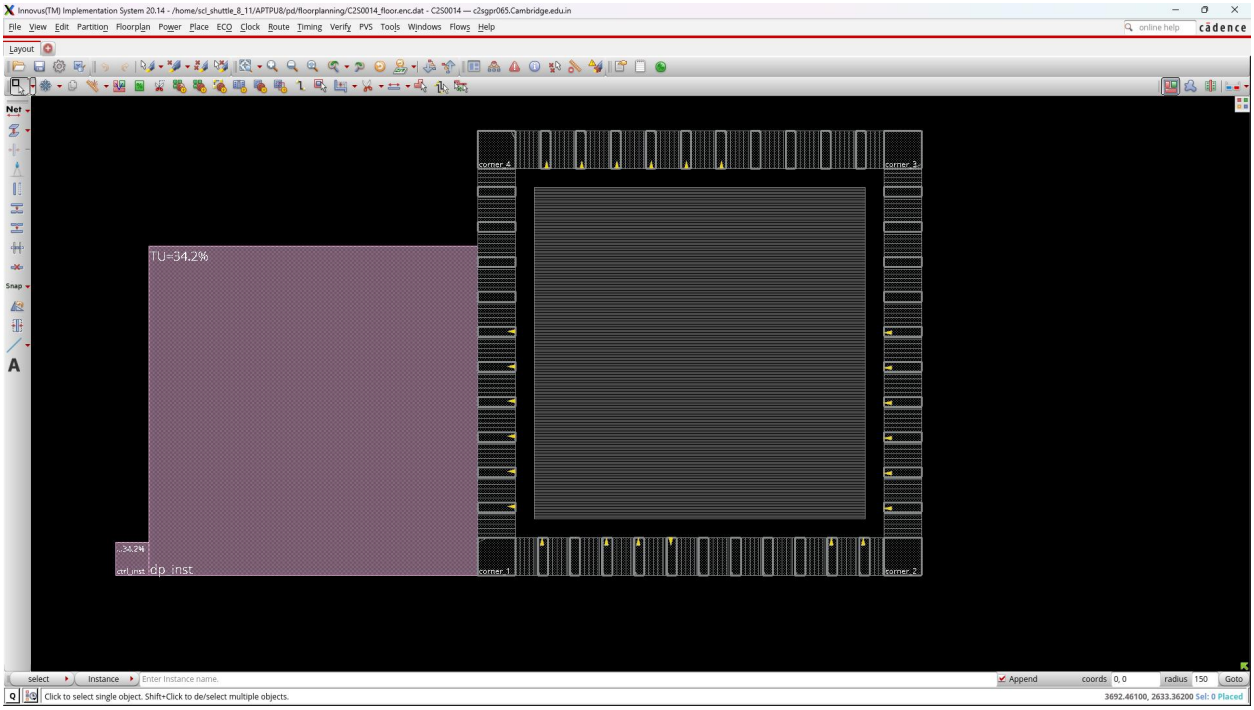
LEC Result: PASS

Gate-Level Simulation (GLS)

Gate-Level Simulation (GLS) was performed using the post-route gate-level netlist to verify the functional correctness of the implemented design after synthesis and routing. The simulation waveform confirms that the clock, control, and data signals operate correctly, and all 144 output results were successfully captured. The obtained Normalized Mean Error Distance (NMED) is 0.0357, with an overall error rate of approximately 3.57%, which is within the acceptable approximation range of the proposed design. These results confirm that the post-route implementation functions as intended and is suitable for signoff.

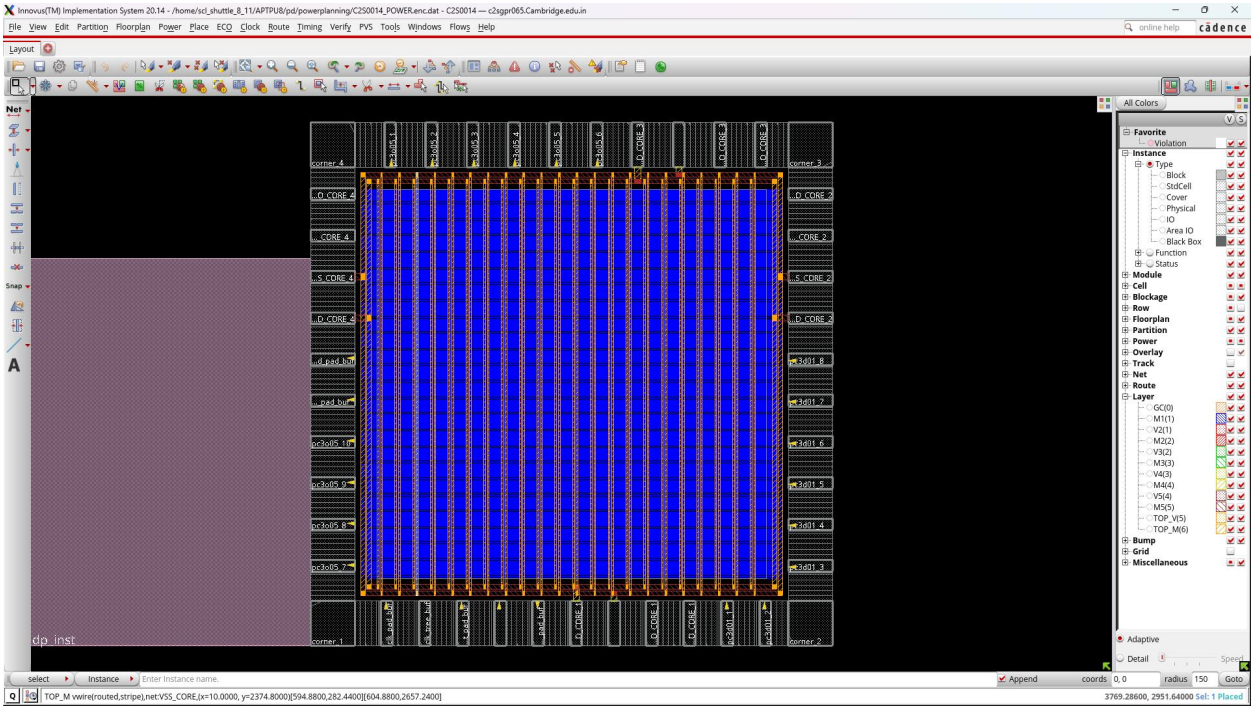
GLS Result: PASS (144/144 outputs captured successfully, Error Rate ≈ 3.57%)

FLOOR PLANNING ANALYSIS:



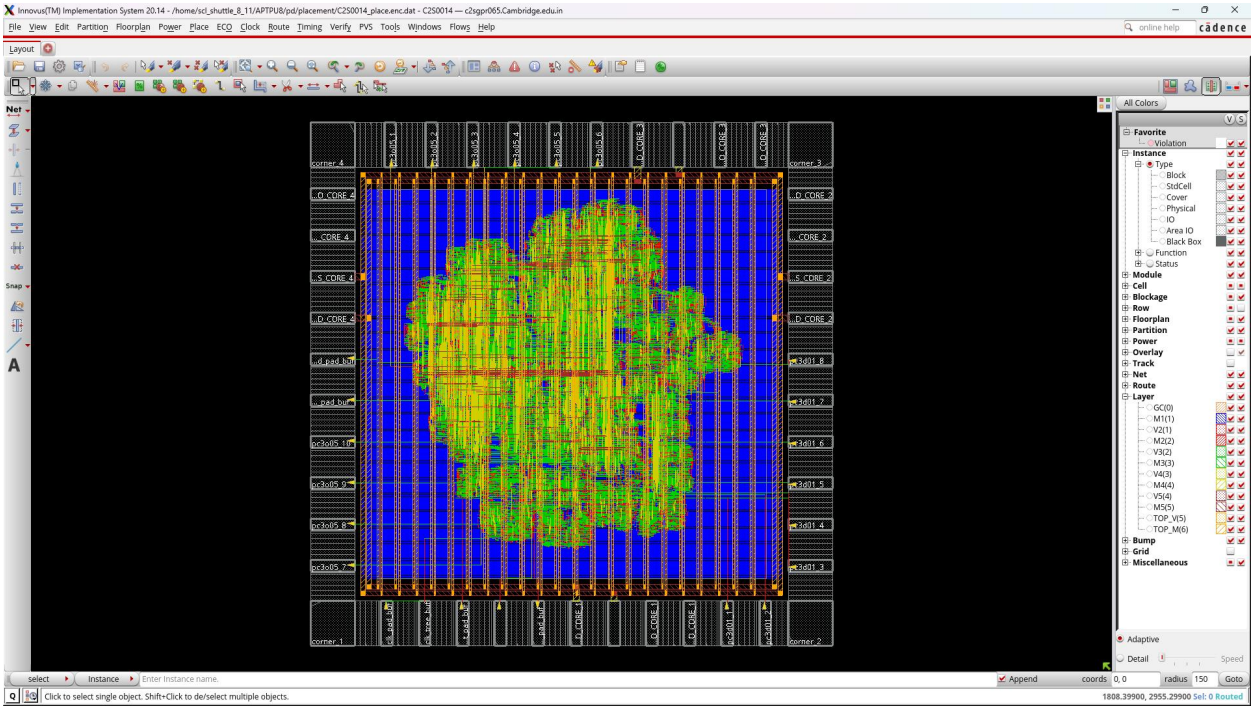
This Figure shows the floorplan of the C2S0014 design generated using Cadence Innovus. During floorplanning, the chip core area, standard cell rows, and I/O pads were arranged to provide an efficient layout for physical implementation. The utilization was maintained at approximately 34.2%, providing sufficient whitespace for placement optimization, clock tree synthesis (CTS), and routing. The corner cells and I/O pads were positioned around the chip boundary, while the core region was reserved for standard cell placement. This floorplan ensures efficient resource utilization, minimizes routing congestion, and facilitates successful placement and routing in the subsequent physical design stages.

POWER PLANNING ANALYSIS:



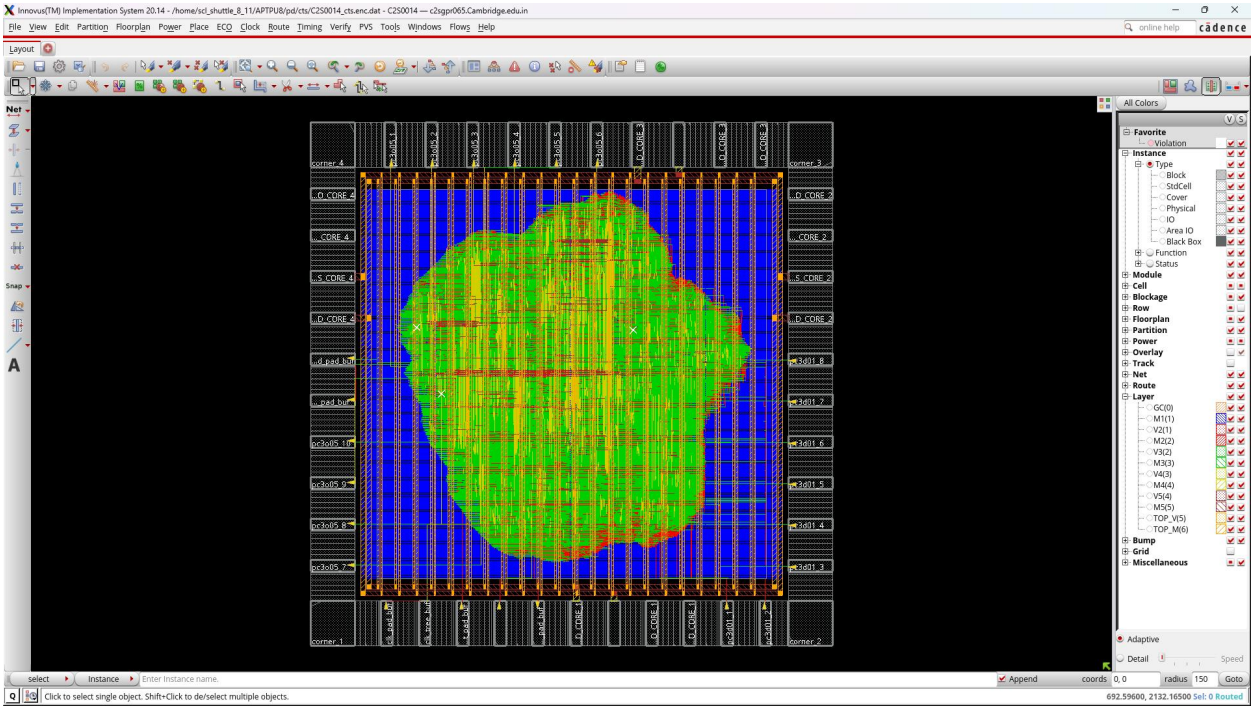
This Figure illustrates the power planning of the C2S0014 design implemented in Cadence Innovus. During this stage, dedicated VDD and VSS power rings were created around the core, and horizontal and vertical power stripes were inserted across the design to distribute power uniformly. The generated power grid provides a low-resistance path for power delivery, minimizing IR drop and improving power integrity. This ensures that all standard cells receive a stable power supply, enabling reliable operation during placement, clock tree synthesis (CTS), routing, and signoff verification.

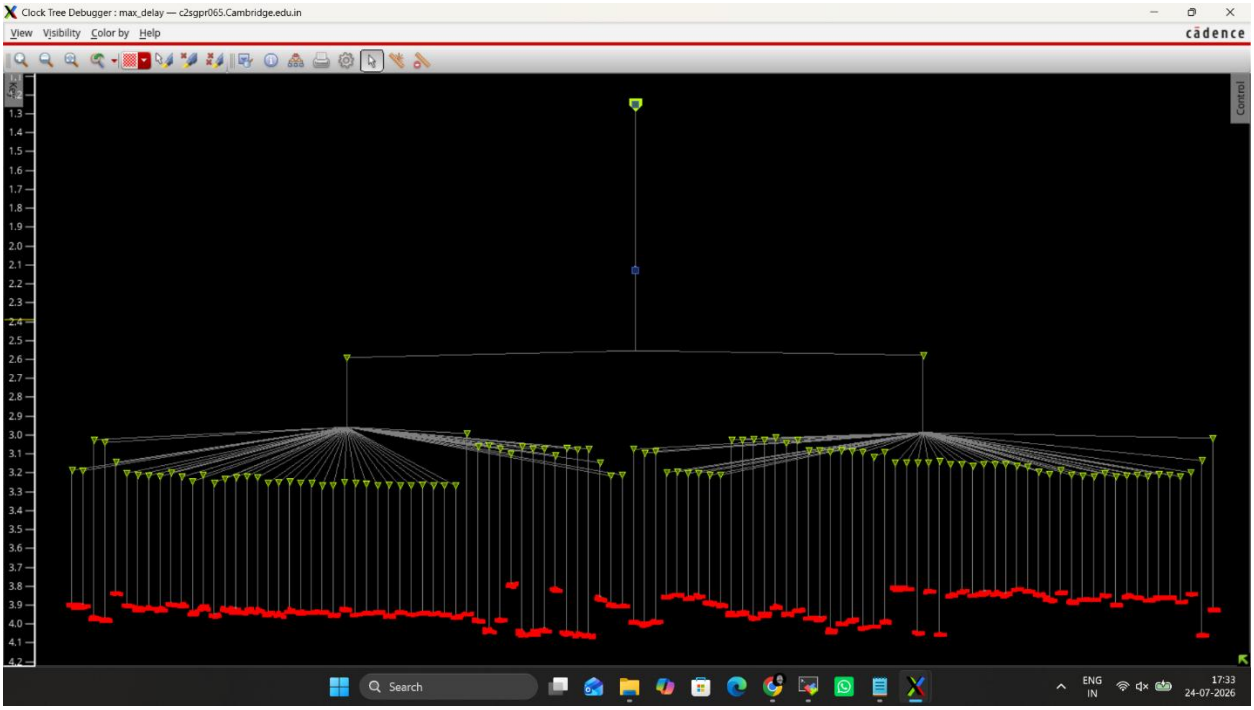
PLACEMENT ANALYSIS:



This Figure shows the placement stage of the **C2S0014** design performed in **Cadence Innovus**. During placement, the synthesized standard cells are automatically arranged within the core area while considering timing, power, and routing constraints. The placement tool optimizes the cell locations to minimize wire length, reduce routing congestion, and improve timing performance. The power rings and power stripes generated during the power planning stage are preserved, while the standard cells are distributed efficiently across the core. A well-optimized placement provides the foundation for successful clock tree synthesis (CTS) and routing by ensuring balanced cell distribution and improved design performance.

Clock Tree Synthesis (CTS) ANALYSIS:





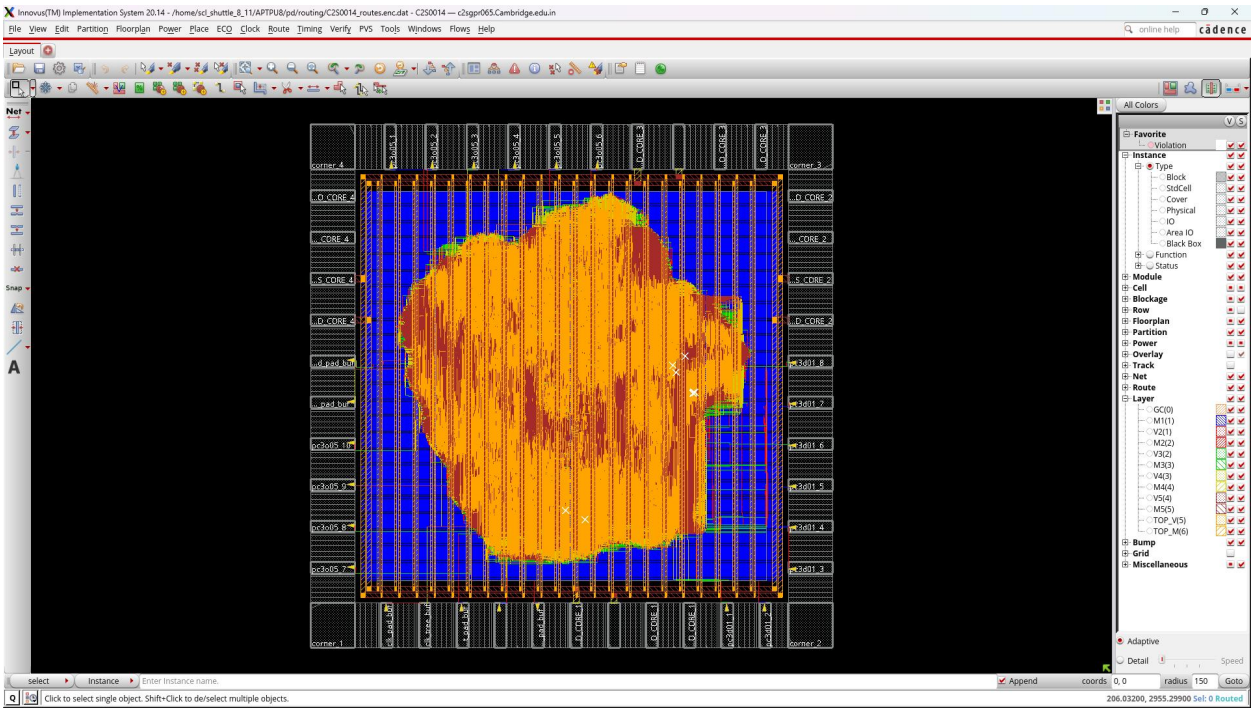
Clock Tree Synthesis (CTS) is one of the most important stages in the ASIC physical design flow. The primary objective of CTS is to distribute the clock signal from the clock source to all sequential elements, such as flip-flops and registers, while minimizing clock skew, insertion delay, and clock uncertainty. A balanced clock network ensures that all sequential elements receive the clock signal with minimal timing variation, thereby improving the overall timing performance and reliability of the design.

During the CTS process, dedicated clock buffers and inverters are inserted automatically by the EDA tool to construct an optimized clock distribution network. The clock tree is designed to provide a balanced path from the clock source to every clock sink while satisfying setup and hold timing requirements. Proper clock distribution also reduces timing violations and improves overall circuit performance.

The first figure illustrates the physical layout of the APTPU after the completion of Clock Tree Synthesis. The inserted clock buffers and optimized routing are integrated into the design while maintaining efficient utilization of the available chip area. The second figure presents the generated clock tree structure, showing the hierarchical distribution of the clock signal from the clock source to multiple clock endpoints. The branching structure demonstrates how the clock network is balanced to achieve uniform clock arrival times across the design.

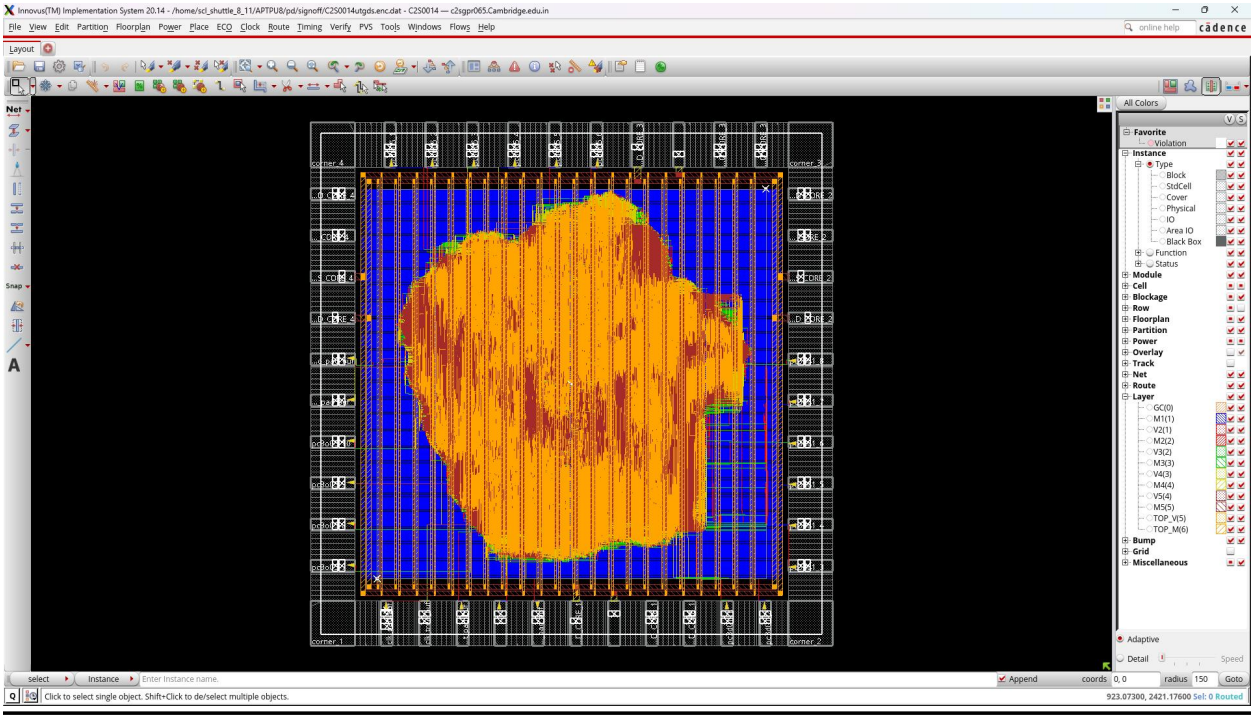
The successful completion of Clock Tree Synthesis provides a robust clock distribution network, enabling the design to proceed to the routing and signoff stages while meeting the required timing constraints.

ROUTING ANALYSIS:



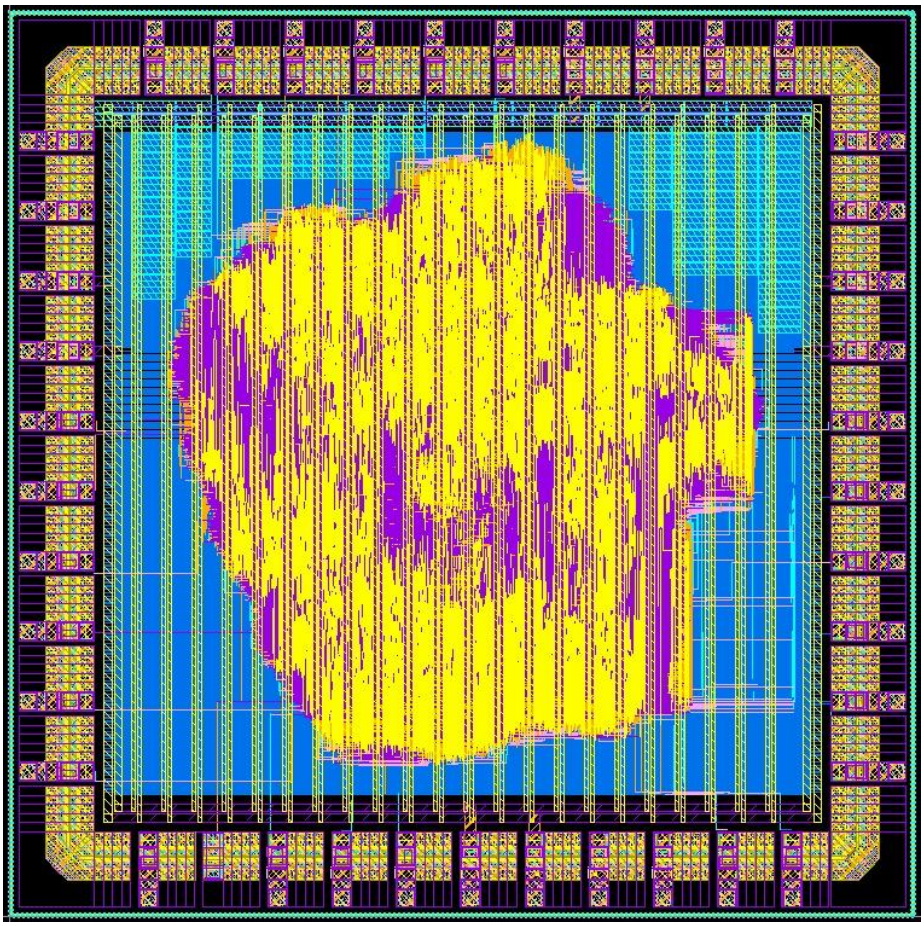
This Figure illustrates the routing stage of the C2S0014 design in Cadence Innovus. During routing, all signal nets were connected using the available metal layers while satisfying design rule constraints and timing requirements. The routing tool optimized the interconnections to minimize wire length, reduce delay, and avoid routing congestion. The previously generated power grid and clock network were preserved, ensuring reliable signal propagation throughout the design. Successful completion of routing confirms that all required signal connections were established, preparing the design for post-route optimization, timing verification, and physical signoff. across the design, improving timing performance and ensuring reliable synchronization of all flip-flops. The optimized CTS layout serves as the foundation for the final routing and timing signoff stages.

GDS Layout (Signoff) ANALYSIS:



This Figure shows the final **GDS layout** of the **C2S0014** design generated using **Cadence Innovus** after completing the physical design flow. The layout includes the placed standard cells, clock network, routed signal interconnections, power distribution network, I/O pads, and corner cells. At this stage, all design rule violations are resolved, and the layout satisfies the required timing, routing, and physical constraints. The finalized GDSII database represents the complete physical implementation of the design and is ready for fabrication after successful signoff verification, including DRC, LVS, and post-layout validation.

Final Tapeout Layout:



The figure below shows the **final tapeout layout** of the **Approximate Computing Based Tensor Processing Unit (APTPU)** generated after the complete ASIC physical design flow. The design has been successfully implemented and verified using **Cadence Virtuoso**, ensuring that the layout accurately represents the synthesized netlist and satisfies the required design constraints. The layout includes the core logic, I/O pads, power distribution network, routing resources, and interconnections required for fabrication. After successful verification, the design is considered **tapeout-ready**, indicating that it is prepared for semiconductor manufacturing.

CONCLUSION:

The **Approximate Tensor Processing Unit (APTPU)** was successfully designed, verified, synthesized, and implemented using a complete ASIC design flow. The project demonstrates the application of approximate computing techniques to achieve efficient tensor processing while reducing hardware complexity, power consumption, and silicon area. Functional verification confirmed the correct operation of the design, and physical implementation was completed successfully through floorplanning, placement, Clock Tree Synthesis (CTS), routing, and tapeout generation. Logical Equivalence Checking (LEC) and timing verification ensured that the synthesized and physical designs were functionally equivalent to the RTL design. The final tapeout layout is fabrication-ready and satisfies the required design constraints. This project provides practical experience in the complete RTL-to-GDSII ASIC design flow and demonstrates the effectiveness of approximate computing for energy-efficient AI hardware accelerators.

Project Highlights:

- Designed and implemented an **Approximate Tensor Processing Unit (APTPU)** for AI acceleration using **Verilog HDL**.
- Successfully verified the design through **functional simulation**, **SpyGlass (Lint, CDC, RDC)**, and **Logical Equivalence Checking (LEC)**.
- Completed the complete **ASIC RTL-to-GDSII design flow**, including synthesis, floorplanning, placement, Clock Tree Synthesis (CTS), routing, and physical verification.
- Generated a **functionally correct, timing-clean, and tapeout-ready ASIC layout** suitable for fabrication.
- Gained hands-on experience with **Cadence** and **Synopsys** EDA tools used in modern VLSI design.
- **Successfully completed the entire APTPU ASIC implementation—from RTL design to final tapeout—within just 10 days**, demonstrating efficient project execution and strong understanding of the ASIC design flow